

M18 - Project Handbook

HOW TO BUILD A DATA ENGINEERING PROJECT FROM ZERO

Imagine you joined a small team

Your manager will not say: "write line 1, then line 2." They will say: "orders are wrong after reruns" or "we need trusted daily revenue." Your job is to turn that messy request into a safe, testable pipeline. This handbook teaches that translation step by step.

Part 1 - What a project actually is

A portfolio project is not a folder full of tools. It is evidence that you can understand a business problem, create a data contract, build a repeatable flow, detect bad data, recover from failure and explain your choices.

The universal 12-phase cycle

Phase	What you are deciding	Artifact
1. Decode the business problem	Translate the brief into concrete questions, outputs, freshness needs and failure consequences.	business_questions.md or project notes
2. Profile every source	Find row grain, business key, timestamps, nulls, duplicates, late-data and schema risks.	source_profile/evidence
3. Write source contracts	State required/optional fields, types, keys, freshness and schema-change behavior.	source_contracts.md
4. Choose trusted output grain	Write one sentence defining what one trusted row means before transforms.	grain statement + key list
5. Draw the smallest correct architecture	Show source -> raw/landing -> validation -> trusted -> serving, plus orchestration/evidence.	architecture.md/diagram
6. Preserve raw evidence	Land unchanged source payload/file plus batch metadata so you can replay/debug.	raw folder + metadata
7. Validate and quarantine	Required keys, types/ranges, duplicate rules and relationship checks before trusted publish.	quality report + quarantine
8. Transform/model deliberately	Create trusted model with explicit grain, keys and reconciled measures.	SQL/dbt/Python/Spark model
9. Make reruns safe	Define idempotency key, incremental rule/watermark, late data and backfill behavior.	rerun/backfill evidence
10. Orchestrate and test failure	Run repeatable dependency order; deliberately trigger one supplied failure and save first failing evidence.	task logs/checker/test output
11. Repair and fresh-retry	Name root cause, apply smallest safe repair, rerun failed path + end-to-end controls.	repair note + new evidence
12. Publish and defend	README, runbook, architecture, limitations, evidence and 5-minute verbal defense.	portfolio-ready repo

Part 2 - A tiny worked example before the real projects

Toy business

A fruit shop drops one orders.csv each morning. The owner wants daily revenue. Sometimes the same file is sent twice and one row may have a missing order_id.

Step A: define one trusted row

Suppose the trusted table grain is **one row per valid order**. That sentence is a contract. If order_id O10 appears twice because the same file was resent, the trusted result should still contain one logical O10 row unless the business says repeated order IDs are legitimate updates.

```
orders.csv
order_id,order_date,amount
010,2026-08-20,100
011,2026-08-20,50
,2026-08-20,75
```

Step B: separate raw, quarantine and trusted

Layer	Rows	Why
raw	all 3 rows exactly as received	evidence/replay
quarantine	missing order_id row	cannot identify trusted order
trusted	O10 + O11	valid business rows

Step C: reconcile

A simple control equation is: **source rows = accepted rows + quarantined rows + deliberately deduplicated rows**. Here: $3 = 2 + 1 + 0$. If you cannot explain where every source row went, your pipeline is not yet trustworthy.

Step D: prove rerun safety

Run the same batch twice. If the trusted table becomes 4 rows (O10/O11 duplicated), the pipeline is not idempotent for this business rule. You need a business key/load strategy such as upsert, merge, processed-file ledger or deduplication - chosen deliberately, not added because the word "idempotency" sounds advanced.

Evidence gate - prove before move-on

- First run: trusted logical rows = 2, revenue = 150.
- Second same-input run: trusted logical rows still = 2, revenue still = 150.
- Raw evidence may contain two deliveries if you intentionally preserve both; trusted business result must follow the stated rerun rule.

STOP - explain before continuing

Why can raw row count increase on a repeated delivery while trusted logical row count should stay stable?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

Part 3 - Grain before code

Grain means: **what exactly does one row represent?** This is the most important modeling sentence in each project. Write it before joins/aggregations.

Possible table	Example grain	Key idea
raw_order_event	one received source record	source file + row/record identity
trusted_order	one business order	order_id
order_line_fact	one product line within an order	order_id + line_no
daily_seller_metric	one seller per calendar day	seller_id + metric_date

Common beginner traps

- Saying "the table contains orders" without saying one row per WHAT.
- Joining two one-to-many tables and then summing money without checking row multiplication.
- Using a generated row number as the only key when the business has a real duplicate/replay rule.

Part 4 - Source contracts: what arrives and what can change

A source contract is your written expectation. It does not require a fancy platform. For each source, record owner, delivery method, grain, business key, required fields, types, timestamp/freshness, duplicate semantics and schema-change policy.

Contract field	Example
source	orders API
grain	one order object
business key	order_id
required	order_id, created_at, amount
optional	coupon_code
freshness	daily by 06:00
duplicates	same order_id may be redelivered; latest updated_at wins
schema change	new nullable field accepted in raw, downstream mapped deliberately

Your source-contract questions

- ___ Who owns this source?
- ___ What does one source row/object mean?
- ___ Which fields identify it?
- ___ Which fields are required?
- ___ Can the same business key arrive again?
- ___ How do I know what is newer?
- ___ What schema change should fail vs be tolerated?

Part 5 - Raw is evidence, trusted is a promise

Raw/landing is not automatically "bad" and trusted is not automatically "clean." The boundary matters because raw preserves source truth while trusted applies your declared business rules.

- Raw: immutable or append-only representation of what was received, plus batch metadata.
- Validated/staging: parsed into usable structure; quality outcomes visible.
- Quarantine/reject: records you intentionally exclude from trusted publish with reason.
- Trusted/Silver: accepted, deduplicated, typed data at a declared grain.
- Gold/mart: business-facing aggregate or model built from trusted data.

Evidence gate - prove before move-on

- Given source batch X, you can show raw evidence.
- Given a rejected row, you can name the exact rule/reason.
- Given a trusted total, you can reconcile it back to accepted source data.

Part 6 - Idempotency, late data and backfill in plain English

Idempotent rerun means repeating the same logical input/work does not incorrectly duplicate/corrupt the trusted result. It does NOT always mean no files/logs change.

Late data means an event for an older business period arrives after the normal processing window. Decide whether it updates old partitions/models, triggers a bounded reprocess, or waits for a correction workflow.

Backfill means intentionally processing a historical range that was missed or needs recomputation. A safe backfill is bounded and does not accidentally reprocess all history or overwrite newer state.

Question	You must be able to answer
Rerun key	What identifies the same logical record/batch?
Update rule	Ignore duplicate? update latest? insert new version?
Watermark	What tells incremental processing where to resume?
Late data	How far back can data arrive and how is it corrected?
Backfill	How do you rerun 2026-08-10..12 only?
Proof	What count/total/state remains correct after rerun?

Part 7 - Failure evidence: do not erase the mistake

A debugging project is stronger when you can show: first failing evidence -> root cause -> smallest safe repair -> rerun proof. Do not delete the failing log because it looks ugly; that log proves you can diagnose.

Bad evidence	Good evidence
"It failed but I fixed it."	Saved error: page 2 JSON malformed; parser stopped before publish.
"Added try/except."	Root cause + which exception is handled + why trusted publish remains blocked.
"Reran successfully."	Failed task rerun + control totals + final clean end-to-end result.

Part 8 - Architecture without cloud-logo decoration

For every architecture box answer five questions: What enters? What leaves? What is persisted? What is trusted? What happens on failure? If you cannot answer them, the box is decoration.

```
SOURCE
-> RAW LANDING (preserve payload + batch metadata)
-> VALIDATE / QUARANTINE
-> TRUSTED MODEL (declared grain)
-> GOLD / SERVING
ORCHESTRATOR controls order/retry.
QUALITY GATES can block publish.
MONITORING observes freshness, failures, quality and cost.
```

STOP - explain before continuing

Pick any box above. What exactly should happen if that box fails after its input was already persisted?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

Part 9 - Project repository that an employer can understand

```
project/  
  README.md  
  architecture.md  
  source_contracts.md  
  runbook.md  
  src/ or sql/ dbt/ dags/ spark/  
  tests/  
  data/           # safe synthetic samples only  
  evidence/  
    attempt-01/  
    incident-01/  
  .env.example    # variable names/placeholders only  
  .gitignore
```

- README: business problem, architecture, setup/run order, outputs, quality, failures, limitations.
- architecture.md: responsibilities/boundaries and key trade-offs.
- runbook.md: rerun, backfill, partial failure, rollback/recovery.
- evidence/: small text/json/screenshots proving tests/results; do not commit huge raw production data.
- tests/: automated code/data/business assertions where practical.

Part 10 - README as a technical story

README section	What a recruiter/engineer should learn
Problem	Who needs what result and why.
Architecture	Simple flow + why each tool/layer exists.
Data contracts/grain	What each source/output row means and keys.
Run	Exact commands/order from clean setup.
Quality/reconciliation	Which bad data is blocked/quarantined and controls.
Rerun/backfill	How duplicate delivery, late data and historical rerun behave.
Failure tested	One real failure + recovery evidence.
Limitations	What is simplified/local/not production-ready.
10x scale	What you would change only if volume/latency justifies it.

Part 11 - Security without pretending to be a security engineer

- Never commit real passwords/tokens/API keys. Use environment variables or secret manager in deployed environment.
- Treat PII as part of the data design: identify it, restrict access, mask/minimize where possible.
- Least privilege: reporting roles should not own/drop/write raw production data unless required.
- Logs must not print secrets or unnecessarily dump PII.
- Use synthetic data in public GitHub projects.

Part 12 - How to defend your project in an interview

A good defense is a chain of cause and evidence, not a tool list.

Problem -> requirement -> design decision -> trade-off -> evidence -> limitation

Example:

"The source can redeliver orders, so order_id is the business key.

I preserve every raw delivery, but trusted load uses that key and update rule.

I proved rerun safety by running the same input twice: trusted count and revenue stayed unchanged.

The limitation is that the local demo uses a file ledger; at higher concurrency I would move state to a transactional metadata store."

Evidence gate - prove before move-on

- Explain trusted output grain in one sentence.
- Explain one rejected design option and why.
- Explain one failure from first error to repair.
- Explain one recovery/backfill decision.
- Explain one security choice.
- Explain one 10x scale change without claiming every tool is needed now.